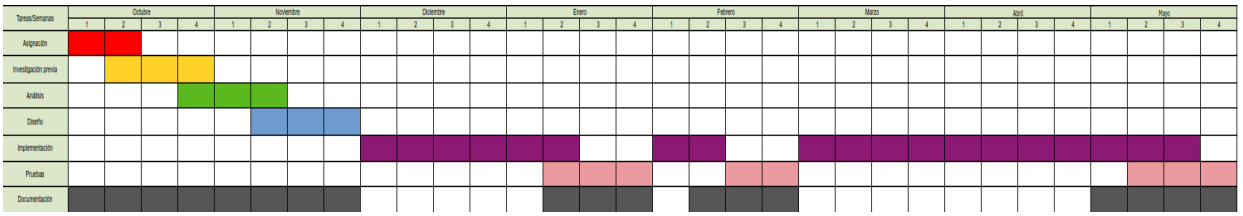
Diseño de Proyectos

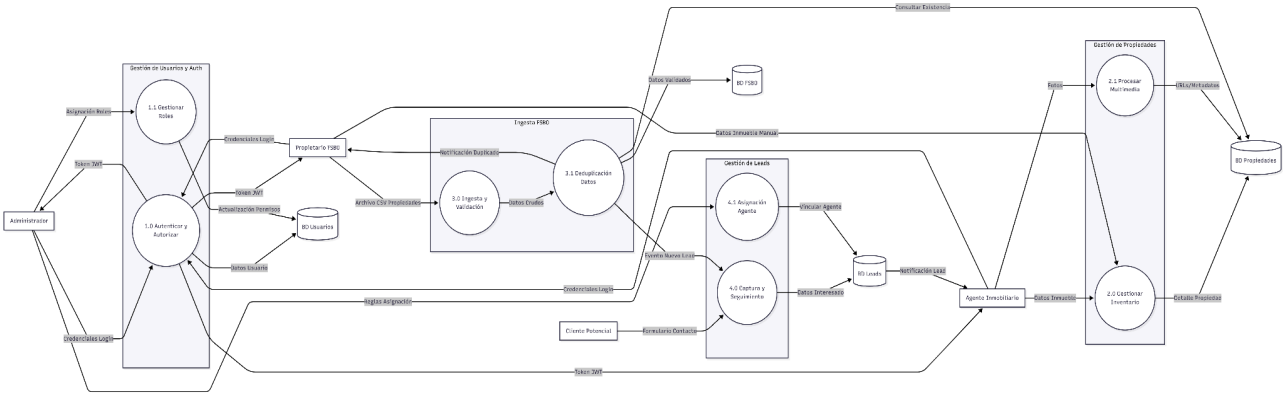
1. Diagrama de Gantt.... 3
2. Diagrama de Flujo de Datos.... 3
3. Diagramas de Flujo.... 4
3.1. Login (auth-service).... 4
3.2. Creación de Inmueble (property-service).... 4
3.3. Registro de Inmuebles desde CSV (fsbo-ingestor-service).... 5
3.4. Consumo de eventos (lead-service).... 5
4. Diagramas de Clases.... 6
5. Diagramas de Secuencia.... 6
5.1. Login del usuario.... 6
5.2. Creación de un Inmueble.... 7
5.3. Refresco del token.... 7
6. Mecanismos de Gestión de Información de la Aplicación.... 7
6.1. ¿Cuál será la información que debe almacenarse como parte del uso de la aplicación?.... 7
6.2. ¿Qué nivel de persistencia tendrá? Es decir, ¿cuánto tiempo debe perdurar en la aplicación?.... 8
6.3. ¿Qué herramientas o tecnologías se emplearán para almacenar dichos datos?.... 8
6.4. ¿Existirán en la aplicación datos de índole sensible? Por ejemplo, datos personales, tarjetas de crédito, etc.... 8
6.5. ¿Qué mecanismos de seguridad se utilizarán para asegurar un tratamiento correcto de los datos?.... 9
7. Diagrama Entidad-Relación.... 9
7.1. auth_db (auth-service).... 10
7.1.1. Estructura General de auth_db.... 11
7.1.2. Tabla users.... 11
7.1.3. Tabla roles.... 12
7.1.4. Tabla user_roles.... 12
7.1.5. Tabla refresh_tokens (opcional).... 12
7.1.6. Relación entre tablas.... 13
7.1.7.Lógica de uso dentro del sistema.... 13
7.1.8. Seguridad aplicada.... 14
7.1.9. Normalización.... 14
7.2. property_db (property-service).... 14
7.2.1. Estructura General de property_db.... 14
7.2.2. Tabla properties.... 15
7.2.3. Tabla property_photos.... 15
7.2.4. Tabla property_features.... 16
7.2.5. Relación entre tablas.... 16
7.2.6. Relaciones Lógicas con otros Microservicios.... 16
7.2.7. Normalización.... 17
7.3. leads_db (lead-service).... 17

7.3.1. Estructura General de leads_db....17
7.3.2. Tabla leads....17
7.3.3. Tabla lead_assignments....19
7.3.4. Tabla lead_events....19
7.3.5. Relaciones entre tablas....20
7.3.6. Relación Lógica con otros Microservicios....20
7.3.7. Normalización....20
7.4. fsbo_db (fsbo-ingestor-service)....20
7.4.1. Estructura General de fsbo_db....21
7.4.2. Tabla fsbo_submission....21
7.4.3. Tabla fsbo_dedup_check....22
7.4.4. Tabla fsbo_errors....22
7.4.5. Relación entre tablas....23
7.4.6. Relación Lógica con otros microservicios....23
7.4.7. Normalización....23
8. Decisiones iniciales para la Interfaz....23
8.1. ¿Qué tipografía o tipografías vas a utilizar?....23
8.1.1. Tipografía Principal (Cuerpo y Datos): Roboto....24
8.1.2. Tipografía Secundaria (Títulos y Encabezados): Montserrat....24
8.2. ¿Cuál será la paleta de colores de la aplicación?....24
8.2.1. Color Primario: Azul Marino Profundo (#1A237E o similares)....24
8.2.2. Color Secundario: Naranja Teja o Coral (#FF7043)....24
8.2.3. Colores Funcionales (Semánticos)....24
8.2.4. Colores Neutros....24
8.3. ¿El diseño planteará la opción de un tema oscuro?....24
9. Logo de la Aplicación....25
10. Mockups....26
10.1. Login....26
10.2. Pantalla Principal de Inicio....27
10.3. Detalles de la Propiedad....28
10.4. Detalles de un Lead....29

1. Diagrama de Gantt

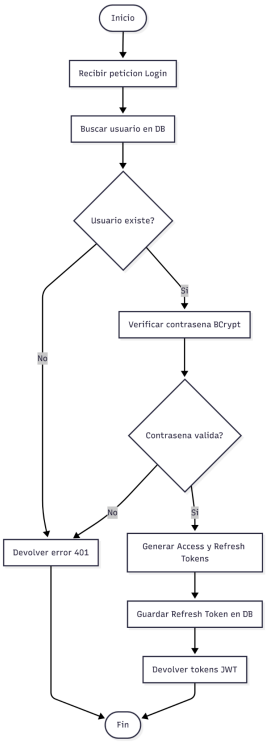


2. Diagrama de Flujo de Datos

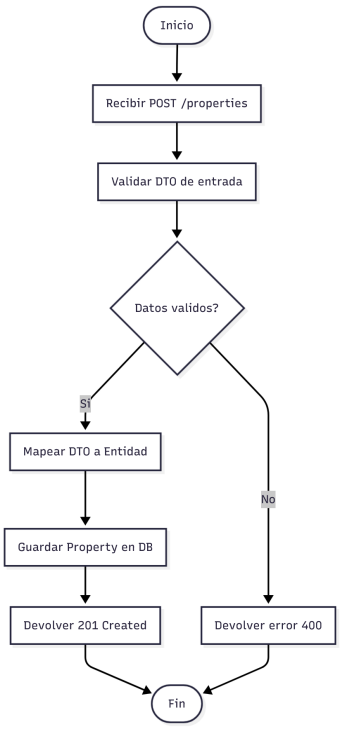


3. Diagramas de Flujo

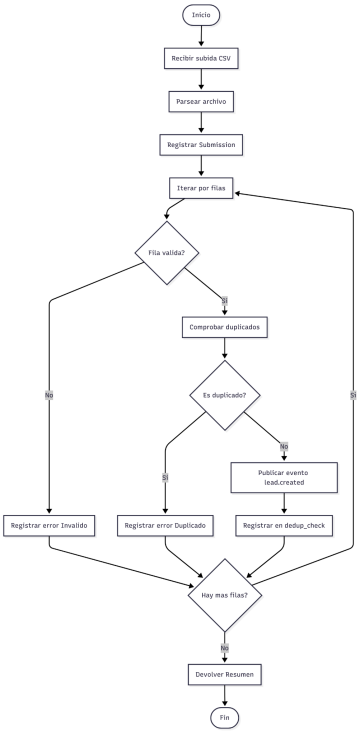
3.1. Login (auth-service)



3.2. Creación de Inmueble (property-service)

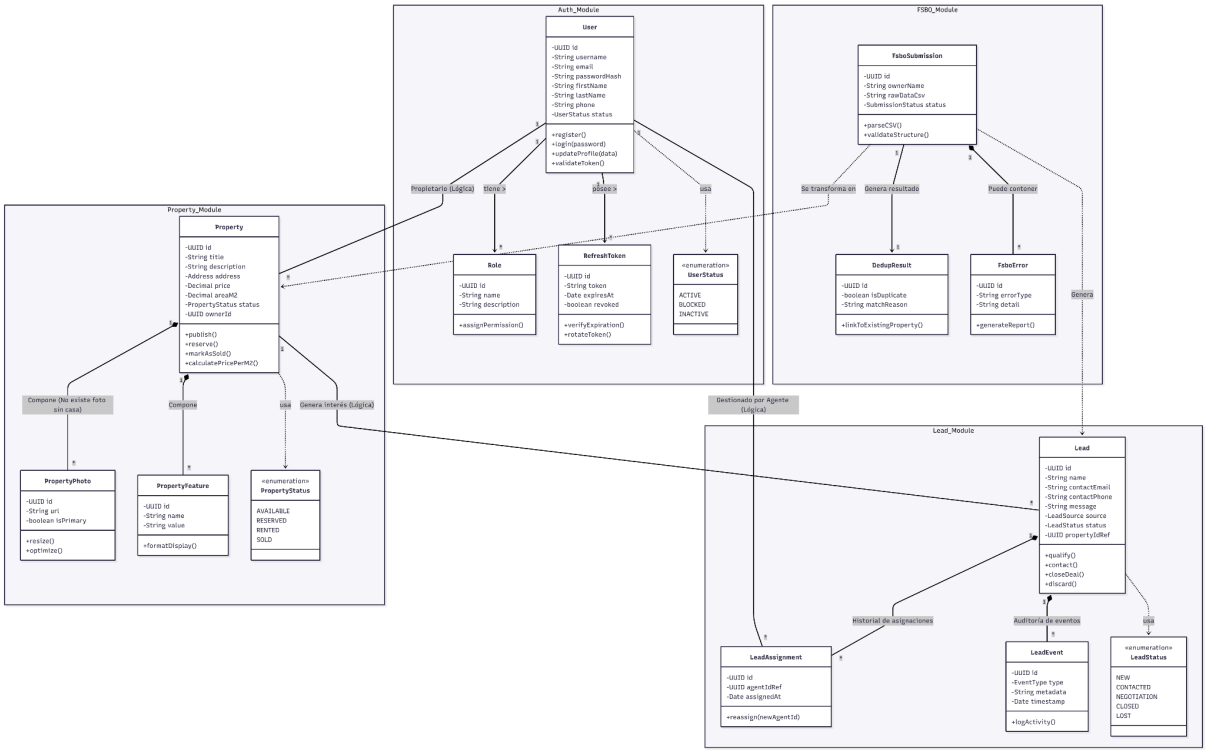


3.3. Registro de Inmuebles desde CSV (fsbo-ingestor-service)



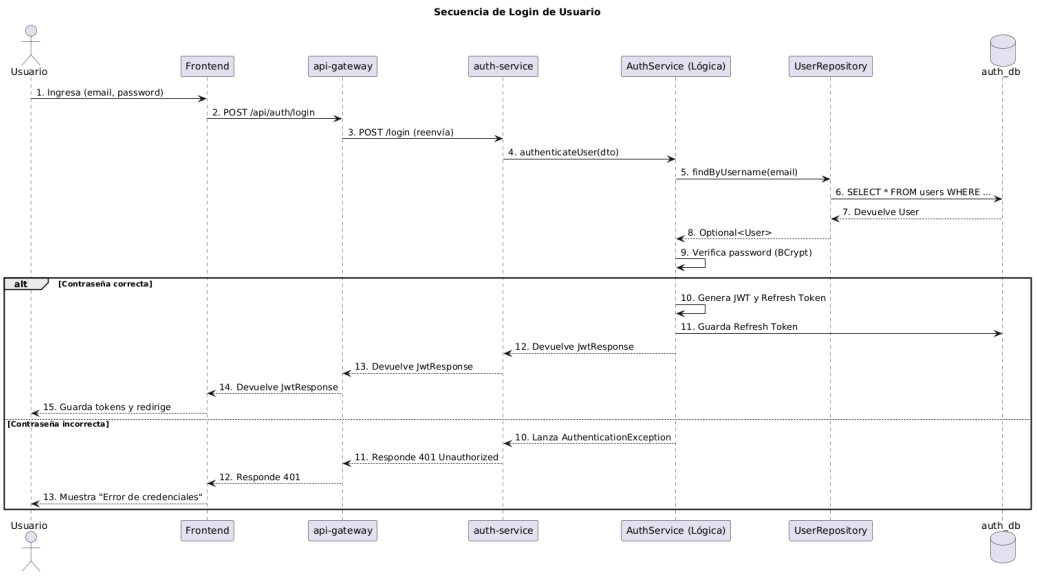
3.4. Consumo de eventos (lead-service)

4. Diagramas de Clases

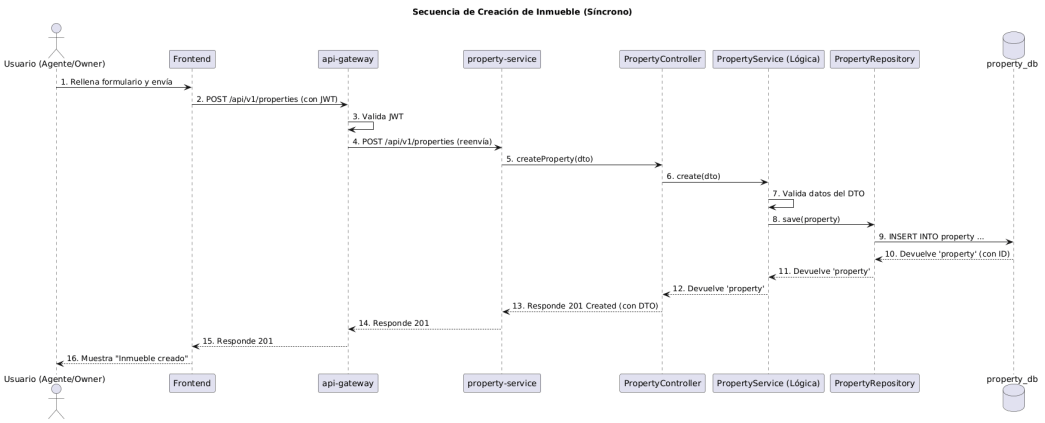


5. Diagramas de Secuencia

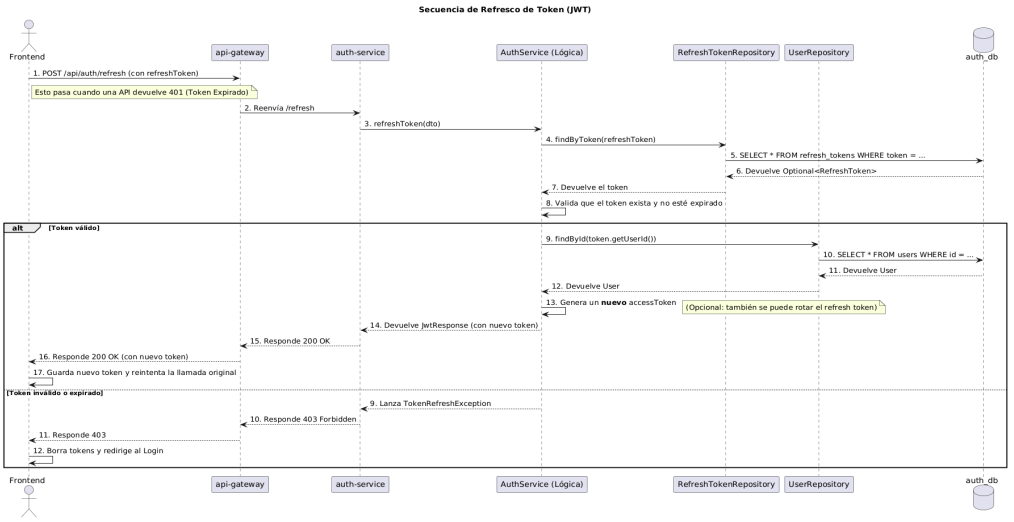
5.1. Login del usuario



5.2. Creación de un Inmueble



5.3. Refresco del token



6. Mecanismos de Gestión de Información de la Aplicación

6.1. ¿Cuál será la información que debe almacenarse como parte del uso de la aplicación?

La aplicación almacenará información crítica para la operación del CRM, dividida por microservicio:

- auth-service: Datos de usuarios (username, email, password hasheada), roles (admin, agent, owner), y tokens de refresco (refresh tokens).

- property-service: Detalles de los inmuebles (título, dirección, precio, m², estado), metadatos de fotos (URLs, si es primaria) y características adicionales (nombre y valor de la característica).

- lead-service: Información de clientes potenciales (nombre, email, teléfono, mensaje), el estado del lead (nuevo, contactado, etc.), la propiedad asociada y las asignaciones a agentes. También almacenará un historial de eventos del lead.

- fsbo-ingestor-service: Registros de las subidas de datos (submissions), errores de ingesta (fsbo_errors) y un registro de comprobación de duplicados (fsbo_dedup_check).

6.2. ¿Qué nivel de persistencia tendrá? Es decir, ¿cuánto tiempo debe perdurar en la aplicación?

El nivel de persistencia varía según el tipo de dato:

- Persistencia Permanente: La gran mayoría de los datos (Usuarios, Roles, Propiedades, Leads, Asignaciones, Submissions) tienen persistencia a largo plazo. Deben perdurar indefinidamente mientras el registro sea relevante para el negocio.

- Persistencia Temporal: Los refresh_tokens son persistentes, pero tienen una fecha de caducidad (expires_at). Una vez caducados, pueden ser eliminados por un proceso de limpieza.

- Volátil (en memoria o de corta duración): Los Access Tokens (JWT) son emitidos pero no se almacenan persistentemente en la base de datos. Su persistencia es la duración de su tiempo de vida.

6.3. ¿Qué herramientas o tecnologías se emplearán para almacenar dichos datos?

Las tecnologías principales para la persistencia de datos en el backend son:

- Base de Datos: PostgreSQL es la base de datos relacional elegida.

- Framework de Acceso: Spring Data JPA/Hibernate como proveedor de ORM.

- Gestión de Migraciones: Flyway se usará para gestionar las migraciones de esquema de la base de datos de forma versionada por cada microservicio.

6.4. ¿Existirán en la aplicación datos de índole sensible? Por ejemplo, datos personales, tarjetas de crédito, etc.

Sí, la aplicación manejará datos personales sensibles:

- Datos de Usuarios: email , password_hash, first_name , last_name y phone.

- Datos de Leads: name , email y phone.

- Datos de FSBO (Propietarios): owner_name , owner_email y owner_phone.

6.5. ¿Qué mecanismos de seguridad se utilizarán para asegurar un tratamiento correcto de los datos?

Se implementará una estrategia de seguridad robusta en varias capas:

- Autenticación y Sesión: Autenticación centralizada en auth-service mediante JWT (Access y Refresh Tokens).

- Protección de contraseñas: Almacenamiento de contraseñas usando un hash fuerte, específicamente BCrypt.

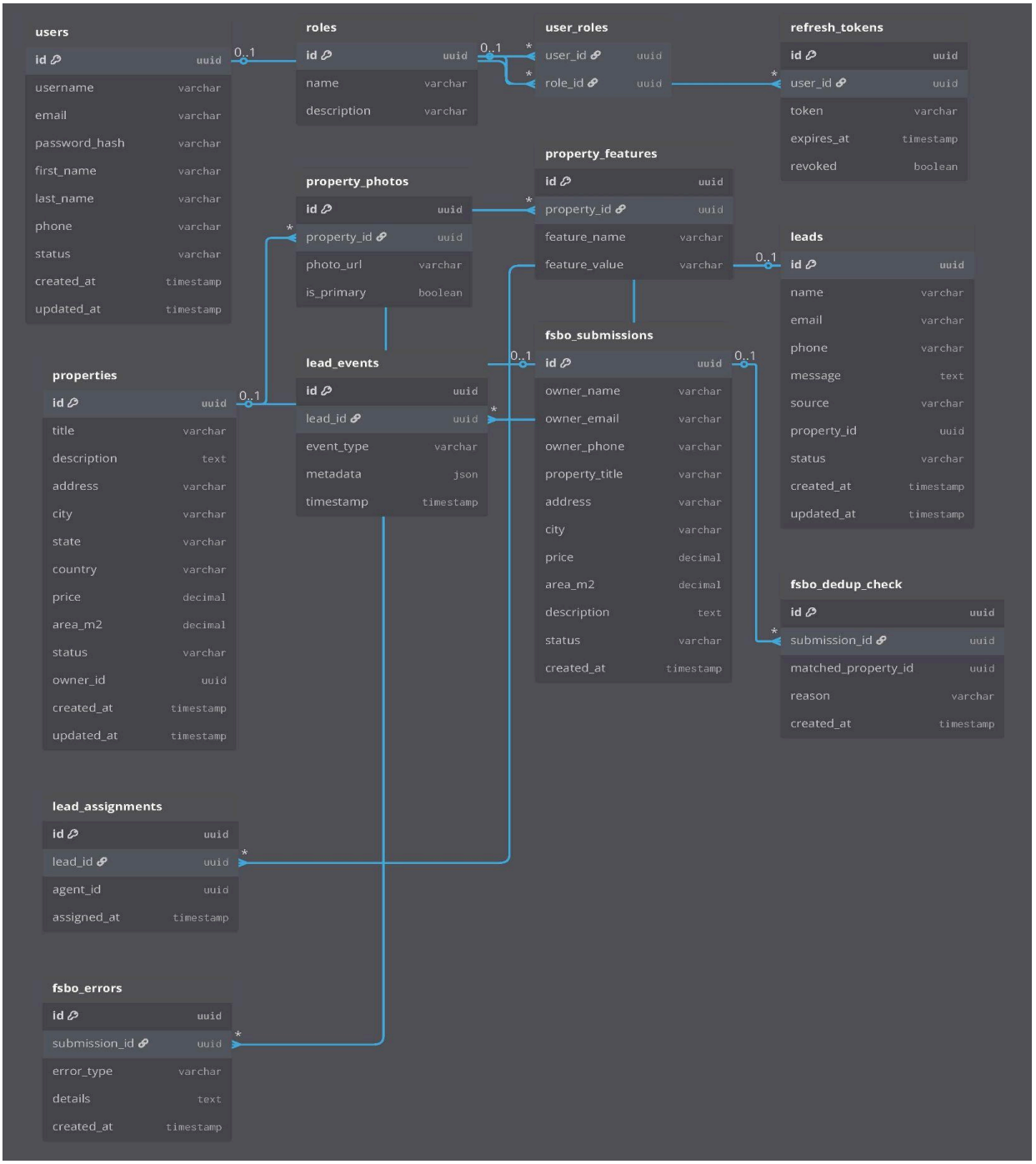
- Seguridad en el Gateway: El API Gateway validará el JWT en cada petición entrante, evitando que peticiones no autorizadas lleguen a los microservicios internos.

- Autorización: Se usarán Roles (ROLE_ADMIN, ROLE_AGENT, ROLE_OWNER) y seguridad a nivel de método con anotaciones (@PreAuthorize) para proteger endpoints específicos.

- Seguridad en Tránsito: Se requerirá HTTPS en producción para cifrar la comunicación.

- Gestión de Secretos: Las contraseñas de bases de datos y otros secretos no estarán en el código, sino que se gestionarán mediante variables de entorno.

7. Diagrama Entidad-Relación



7.1. auth_db (auth-service)

El auth-service es el microservicio encargado de:

- Autenticar usuarios (login con JWT).
- Registrar y gestionar cuentas (admins, agentes, propietarios, clientes).
- Asignar roles y permisos a cada usuario.
- Emitir y validar tokens de acceso y de refresco.

En la arquitectura del proyecto basada en microservicios:

- El api-gateway valida el JWT generado por este servicio antes de dejar pasar peticiones.
- Los demás microservicios (property, leads, fsbo) confían en el JWT y en los roles definidos aquí.

Por eso la base de datos (auth_db) es pequeña, pero crítica: contiene toda la identidad del sistema.

7.1.1. Estructura General de auth_db

Tabla	Descripción
users	Datos personales y credenciales de los usuarios.
roles	Define los distintos roles del sistema
user_roles	Une usuarios con roles
refresh_tokens	Almacena tokens de sesión prolongada (opcional)

7.1.2. Tabla users

Contiene toda la información de identidad de los usuarios del sistema (administrador, agentes, propietarios, FSBO, leads, etc).

Campo	Tipo	Descripción
id	id	Identificador único de usuarios (PK)
username	varchar	Nombre de usuario
email	varchar	Correo electrónico único para login
password_hash	varchar	Contraseña cifrada con BCrypt
first_name	varchar	Nombre real del usuario
last_name	varchar	Apellido(s) real del usuario
phone	varchar	Télefono de contacto
status	varchar	Estado: ACTIVE, INCATIVE, BLOCKED

created_at	timestamp	Fecha de creación del usuario
updated_at	timestamp	Fecha de modificación del usuario

Nota:
- Las contraseñas no se guardan en texto plano, solo en hash BCrypt.
- El email y phone se marcan como únicas para evitar duplicados.
- status permite suspender o reactivar usuarios sin borrarlos.

7.1.3. Tabla roles

Define los diferentes roles del sistema, necesarios para el control de acceso (autorización).

Campo	Tipo	Descripción
id	id	Identificador único del rol (PK)
name	varchar	Nombre del rol: ADMIN, AGENT, OWNER, CLIENT.
descripción	varchar	Descripción legible (ej. “Usuario con privilegios de administrador”).

7.1.4. Tabla user_roles

Es una tabla intermedia que asocia usuarios con sus roles (relación muchos-a-muchos).

- Un usuario puede tener varios roles (ej. un AGENTE que también es ADMIN).
- Un rol puede estar asignado a muchos usuarios.

Campo	Tipo	Descripción
user_id	id	Referencia al usuario (user.id)
role_id	id	Referencia al rol (rol.id)

Nota:
- Clave primaria compuesta (user_id y role_id).

7.1.5. Tabla refresh_tokens (opcional)

Guarda tokens de refresh para sesiones persistentes (cuando JWT expira, el usuario puede renovarlo sin volver a loguearse).

Campo	Tipo	Descripción
id	id	Identificador único del token
user_id	id	Usuario al que pertenece
token	varchar	Valor del token de refresco
expires_at	timestamp	Fecha de expiración
revoked	boolean	Indica si fue invalidado manualmente

Funcionamiento:

- El usuario hace login → se genera un access token (JWT) + refresh token.
- Cuando el JWT expira, el cliente envía el refresh token al auth-service.
- Si es válido y no esta revocado → se emite un nuevo JWT.

Esto mejora la seguridad y evita pedir login constantemente.

7.1.6. Relación entre tablas

users (1) —— (N) user_roles (N) —— (1) roles
users (1) —— (N) refresh_tokens

7.1.7.Lógica de uso dentro del sistema

Acción	Tablas que intervienen	Qué hace el sistema
Registrar usuario	users, user_roles	Crea un registro en users, asigna un rol predeterminado (CLIENT or OWNER).
Login	users, refresh_tokens	Verificar email+ password hash, genera JWT y refresh token.
Validar permisos	users, user_roles	Comprueba si el usuario tiene el rol adecuado.
Asignar rol	user_roles	El admin puede agregar o quitar roles.
Cerrar sesión global	refresh_tokens	Marca token como revocado.

7.1.8. Seguridad aplicada

- Contraseñas: cifradas con BCrypt.
- Tokens JWT: firmados con clave secreta o RSA.
- Roles: verificados por Spring Security usando anotaciones como
- @PreAuthorize(”hashRole(’ADMIN’)”).
- Usuarios inactivos: status ≠ ‘ACTIVE’ → se bloquea autentificación.

7.1.9. Normalización

3° Forma normal:

- Cada tabla tiene un propósito único.
- No hay campos redundantes.
- Las relaciones están bien separadas (sin dependencias transitivas).

7.2. property_db (property-service)

El property-service es el microservicio encargado de gestionar los inmuebles, tanto los publicados por la inmobiliaria (a través de los agentes) como los subidos por propietarios particulares (FSBO).

Sus responsabilidades son:

- CRUD completo de propiedades (crear, leer, actualizar, eliminar).
- Manejar los datos principales de un inmueble (dirección, precio, superficie, estado).
- Guardar las fotos y características (features) asociadas.
- Mantener el estado del inmueble (publicado, reservado, alquilado).
- Servir como fuente de información para otros servicios como lead-service o fsbo-ingestor .

7.2.1. Estructura General de property_db

Tabla	Descripción breve
properties	Información principal de cada inmueble
property_photos	Almacena las fotos asociadas a una propiedad.

property_features	Contiene las características adicionales (número de habitaciones, baños, etc.).

7.2.2. Tabla properties

Almacena toda la información principal de los inmuebles gestionados en el sistema.

Campo	Tipo	Descripción
id	uuid	Identificador único del inmueble (PK).
title	varchar	Título o nombre corto de la propiedad.
description	text	Descripción detallada.
address	varchar	Dirección completa.
city	varchar	Ciudad donde se ubica.
state	varchar	Provincia o estado.
country	varchar	País.
price	decimal	Precio de venta o alquiler.
area_m2	decimal	Superficie del inmueble (en metros cuadrados).
status	varchar	Estado del inmueble: AVAILABLE , RESERVED , RENTED , SOLD .
owner_id	uuid	ID del propietario, referencia lógica a auth-service.users.id.
created_at	timestamp	Fecha de creación.
updated_at	timestamp	Fecha de última modificación.

Nota:

- owner_id no es foreign key física, sino una referencia lógica al usuario
- propietario (porque auth-service tiene su propia base de datos).
- status permite controlar la disponibilidad visible en el portal.
- price y area_m2 son DECIMAL para precisión (no float).

7.2.3. Tabla property_photos

Su propósito es guardar las URLs o rutas de las fotos de cada inmueble.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK).
property_id	uuid	ID de la propiedad asociada (properties.id).
photo_url	varchar	URL o ruta del archivo de imagen (puede apuntar a un bucket, S3, etc.).
is_primary	boolean	Indica si es la foto principal del inmueble.

7.2.4. Tabla property_features

Su propósito es permitir añadir atributos personalizados a cada propiedad (flexible y extensible).

Por ejemplo: número de habitaciones, baños, tipo de calefacción, garaje, etc.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK).
property_id	uuid	ID de la propiedad (properties.id).
feature_name	varchar	Nombre del atributo (ej. “habitaciones”).
feature_value	varchar	Valor del atributo (ej. “3”).

7.2.5. Relación entre tablas

properties (1) ---- (N) property_photos
properties (1) ---- (N) property_features

7.2.6. Relaciones Lógicas con otros Microservicios

Origen	Destino	Tipo	Descripción
auth-service	property-service	Lógica	owner_id referencia al usuario propietario (UUID).
lead-service	property-service	Lógica	property_id en los leads apunta a una propiedad.
fsbo-ingestor-service	property-service	Lógica (por deduplicación)	Compara direcciones o teléfonos para evitar duplicados.

7.2.7. Normalización

3a forma normal:

- Cada tabla tiene una sola responsabilidad.
- No hay datos redundantes.
- Las relaciones 1:N están correctamente modeladas (property → photos , property → features).

7.3. leads_db (lead-service)

El lead-service es el microservicio encargado de:

- Registrar interesados que contactan por una propiedad.
- Gestionar el ciclo de vida del lead: NEW → CONTACTED → NEGOTIATION → CLOSED → LOST
- Asignar leads a agentes (manual o automático).
- Registrar historial de eventos (auditoría / tracking).
- Consumir eventos de fsbo-ingestor-service (lead.created).
- Producir eventos para integración futura con notificaciones o CRM externos.

7.3.1. Estructura General de leads_db

Tabla	Rol
leads	Información principal del lead/interesado.
lead_assignments	Asignación del lead a un agente.
lead_events	Registro de eventos (auditoría y comunicación asíncrona).

7.3.2. Tabla leads

Es la tabla principal, donde se registra cada interesado que entra al sistema, ya sea:

- Desde el formulario web (cliente interesado)
- Desde FSBO-ingestor (propietario directo)
- Desde un agente que lo crea manualmente

Campo	Tipo	Descripción
id	uuid	Identificador único del lead (PK).
name	varchar	Nombre del interesado.
email	varchar	Email del interesado.
phone	varchar	Teléfono del interesado.
message	text	Mensaje enviado por el cliente o notas del agente.
source	varchar	Origen del lead: WEB , FSBO , MANUAL , OTHER.
property_id	uuid	Referencia lógica al inmueble relacionado.
status	varchar	Estado del lead (NEW, CONTACTED, NEGOTIATION, CLOSED, LOST).
created_at	timestamp	Fecha en que el lead fue recibido o generado.
updated_at	timestamp	Última fecha en que se modificó el lead.

Nota:

- No existen foreign keys reales hacia property_db ni auth_db.
- Los UUID son referencias lógicas entre microservicios (porque cada microservicio tiene su propia base).
- El estado del lead es clave para el panel del agente.
- source permite diferenciar entre leads que entran desde:
    - Página pública de propiedades
    - Datos importados desde FSBO
    - Creación manual por un agente

7.3.3. Tabla lead_assignments

Registra a qué agente inmobiliario está asignado un lead. Un lead puede estar asignado a un solo agente a la vez, pero puede haber historial de asignaciones si lo reasigna el administrador.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK).
lead_id	uuid	Identificador del lead.
agent_id	uuid	Identificador del agente (referencia lógica a auth_db).
assigned_at	timestamp	Fecha de asignación.

Nota:

- No existe foreign key real hacia auth-service , pero agent_id coincide con users.id .
- Si se reasigna el lead, se crea otro registro, permitiendo auditoría completa.

7.3.4. Tabla lead_events

Registrar todo lo que pasa con un lead:

- lead.created
- lead.assigned
- lead.status_changed
- lead.updated
- lead.imported (desde FSBO)

Campo	Tipo	Descripción
id	uuid	PK
lead_id	uuid	ID del lead relacionado
event_type	varchar	CREATED, ASSIGNED, STATUS_CHANGED, etc.
metadata	json	Información adicional opcional (ej: “assignedTo: agent_id”).
timestamp	timestamp	Momento específico del evento.

7.3.5. Relaciones entre tablas

leads (1) -------- (N) lead_assignments
leads (1) -------- (N) lead_events

7.3.6. Relación Lógica con otros Microservicios

Microservicio	Relación con leads_db
property-service	leads.property_id hace referencia lógica a una propiedad.
auth-service	lead_assignments.agent_id refiere a un usuario con rol AGENT.
fsbo-ingestor-service	Produce eventos lead.created que insertan nuevos leads en leads_db.
gateway	Permite que agentes y admins consulten o modifiquen leads.

7.3.7. Normalización

3a forma normal:

- leads contiene solo datos propios del lead.
- lead_assignments separa asignaciones (histórico).
- lead_events mantiene el registro de auditoría (no duplicado).

7.4. fsbo_db (fsbo-ingestor-service)

Este microservicio tiene cuatro responsabilidades fundamentales:

1. Ingesta de propiedades enviadas por propietarios FSBO:
   - A través de un formulario
   - A través de un archivo CSV

2. Validación de datos:
   - Verificar formato del archivo y campos faltantes
   - Registrar errores

3. Deduplicación:
   - Evitar registrar propiedades duplicadas comparando:
     - Email
     - Teléfono
     - Dirección (opcional)

4. Generación de leads
   - Cuando un FSBO sube un inmueble válido, se genera un evento lead.created, que es consumido por lead-service .

7.4.1. Estructura General de fsbo_db

Tabla	Propósito
fsbo_submissions	Registro principal del inmueble enviado por un propietario FSBO.
fsbo_dedup_check	Resultado del proceso de deduplicación.
fsbo_errors	Errores encontrados durante la validación o carga.

7.4.2. Tabla fsbo_submission

Almacena cada envío realizado por un propietario. Un envío puede venir de:
- Un formulario web
- Un archivo CSV con varias filas (un envío por fila)

Campo	Tipo	Tipo
id	uuid	Identificador único del envío (PK).
owner_name	varchar	Nombre del propietario.
owner_email	varchar	Email del propietario.
owner_phone	varchar	Teléfono del propietario.
property_title	varchar	Título del inmueble.
address	varchar	Dirección principal.
city	varchar	Ciudad.
price	decimal	Precio sugerido.
area_m2	decimal	Metros cuadrados.
description	text	Descripción del inmueble.
status	varchar	PENDING , VALIDATED , REJECTED.
created_at	timestamp	Fecha del registro.

Nota:

- Esta tabla contiene todo lo que el usuario ingresó, incluso si está mal.
- El campo status indica en qué fase está el procesamiento.
- Si los datos son válidos → se crea un lead y la propiedad puede pasar a
- property-service (implementación a futuro).

7.4.3. Tabla fsbo_dedup_check

Registra si un envío FSBO es duplicado o coincide con una propiedad existente. El sistema comprueba:

- Email
- Teléfono
- Dirección similar

Campo	Tipo	Descripción
id	uuid	Identificador único (PK).
submission_id	uuid	ID del envío (fsbo_submissions.id).
matched_property_id	uuid	Referencia lógica a property_service.properties.id.
reason	varchar	Motivo del duplicado: DUPLICATE_EMAIL , DUPLICATE_PHONE , SIMILAR_ADDRESS .
created_at	timestamp	Fecha de la detección.

Nota:

- Esta tabla no impide crear el lead; solo registra por qué fue marcado
- duplicado.
- Puede haber varias coincidencias por envío, por ejemplo:
    - Email duplicado
    - Teléfono duplicado
    - Dirección similar

7.4.4. Tabla fsbo_errors

Registra errores detectados durante la validación del CSV o formulario. Estos errores NO son duplicados; son problemas técnicos o de formato.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK).

submission_id	uuid	Envío al que pertenece.
error_type	varchar	FORMAT , DUPLICATE , MISSING_FIELD , etc.
details	text	Mensaje detallado del error.
created_at	timestamp	Fecha del registro.

7.4.5. Relación entre tablas

fsbo_submissions (1) ---- (N) fsbo_dedup_check
fsbo_submissions (1) ---- (N) fsbo_errors

7.4.6. Relación Lógica con otros microservicios

Origen	Destino
fsbo-ingestor → lead-service	Evento lead.created
fsbo-ingestor → property-service	Comparación lógica para deduplicación
auth-service	Puede existir owner registrado (futuro opcional)

7.4.7. Normalización

3a forma normal:

- No hay duplicación de datos
- Cada tabla tiene una única responsabilidad
- La relación es clara y mantiene trazabilidad

8. Decisiones iniciales para la Interfaz

8.1. ¿Qué tipografía o tipografías vas a utilizar?

Para asegurar una excelente legibilidad en un entorno de gestión de datos (tablas, fichas de clientes, precios) y mantener una estética moderna, se utilizará una combinación de Google Fonts estándar:

8.1.1. Tipografía Principal (Cuerpo y Datos): Roboto

Es la tipografía por defecto en el ecosistema Android/Google. Es extremadamente legible en tamaños pequeños, lo cual es crucial para un CRM donde se muestran muchas listas de propiedades, números de teléfono y precios. Además, sus números son claros y fáciles de diferenciar (importante para importes y superficies).

8.1.2. Tipografía Secundaria (Títulos y Encabezados): Montserrat

Aporta un toque de distinción y modernidad. Funciona muy bien en negrita para los encabezados de las fichas de inmuebles o los títulos de las secciones del Dashboard.

8.2. ¿Cuál será la paleta de colores de la aplicación?

La aplicación necesita color para guiar la atención del usuario (Botones de acción, Estados).

8.2.1. Color Primario: Azul Marino Profundo (#1A237E o similares)

Transmite profesionalidad, seguridad y confianza, valores clave en el sector inmobiliario. Su uso principal será en la barra de navegación, encabezados principales, botones secundarios.

8.2.2. Color Secundario: Naranja Teja o Coral (#FF7043).

Contrasta muy bien con el azul, aporta calidez y dirige la vista a lo importante. Su uso principal será en botones de llamada a la acción, e iconos destacados.

8.2.3. Colores Funcionales (Semánticos)

- Verde (#4CAF50): Para estados positivos.
- Amarillo/Ámbar (#FFC107): Para estados de precaución.
- Rojo (#F44336): Para acciones destructivas o estados negativos.

8.2.4. Colores Neutros

- Fondo: Blanco (#FFFFFF) y Gris Muy Claro (#F5F5F5) para diferenciar áreas del dashboard sin cansar la vista.
- Texto: Gris Oscuro (#212121) para facilitar la lectura.

8.3. ¿El diseño planteará la opción de un tema oscuro?

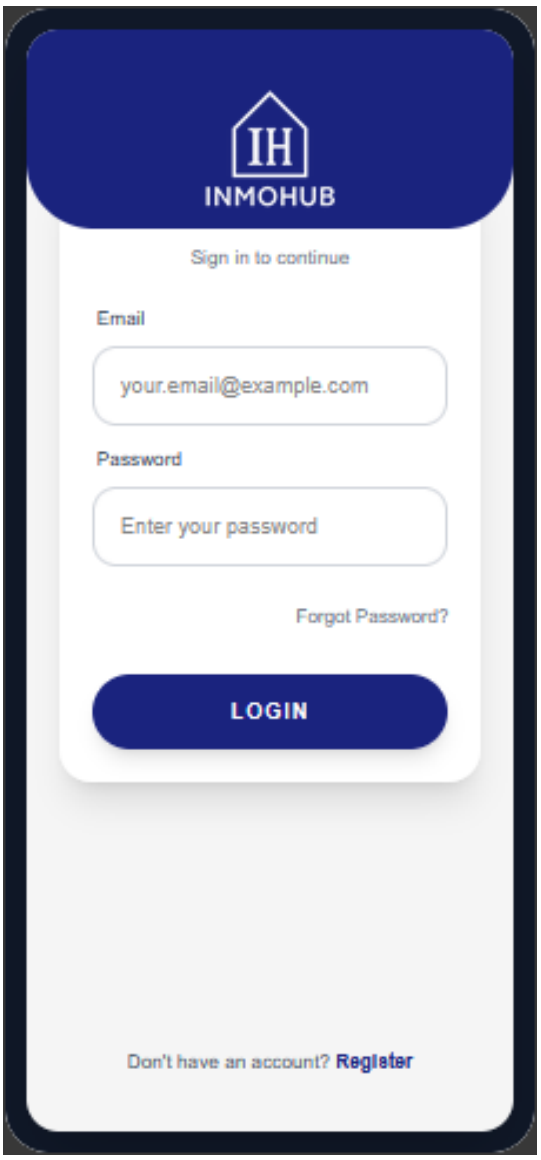
No, en la versión inicial no se implementará el tema oscuro. El desarrollar y testear un tema oscuro duplica el trabajo de diseño de interfaz, ya que hay que verificar el contraste de todos los colores y sombras en ambos modos. Dado el plazo de entrega, se prioriza la funcionalidad.

9. Logo de la Aplicación

Se ha optado por un logo minimalista compuesto por el contorno de una casa que encierra las iniciales “IH”, evocando el sector de bienes raíces de forma clara e inmediata. La tipografía combina estilos clásicos y modernos para transmitir confianza y profesionalidad, utilizando una paleta monocromática que se integra fácilmente en la interfaz de usuario de la aplicación móvil y web.

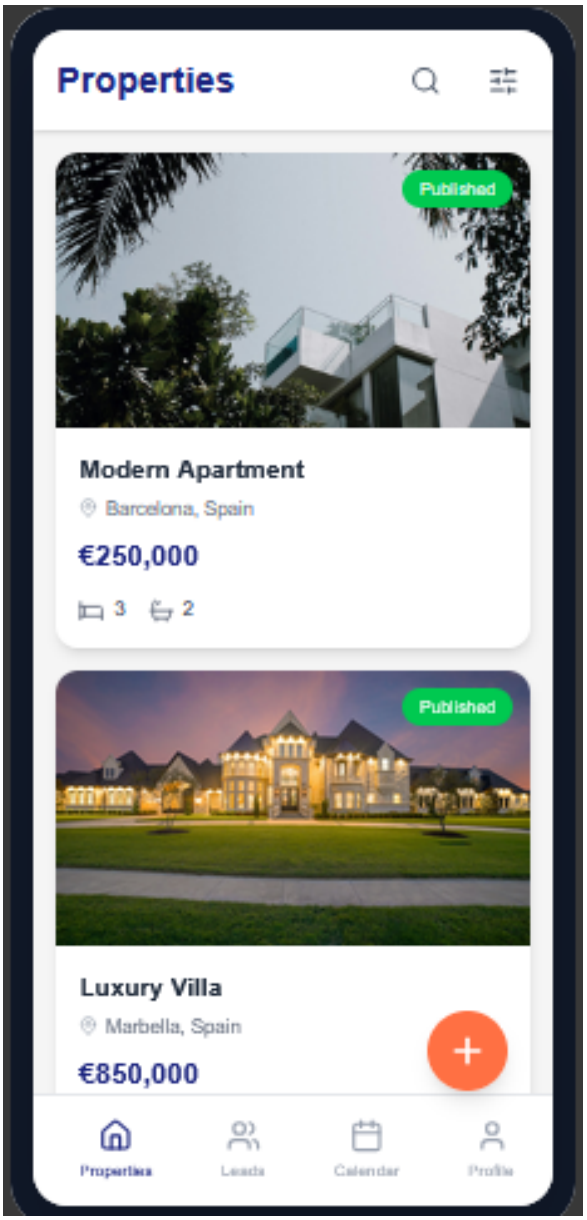
10. Mockups

10.1. Login



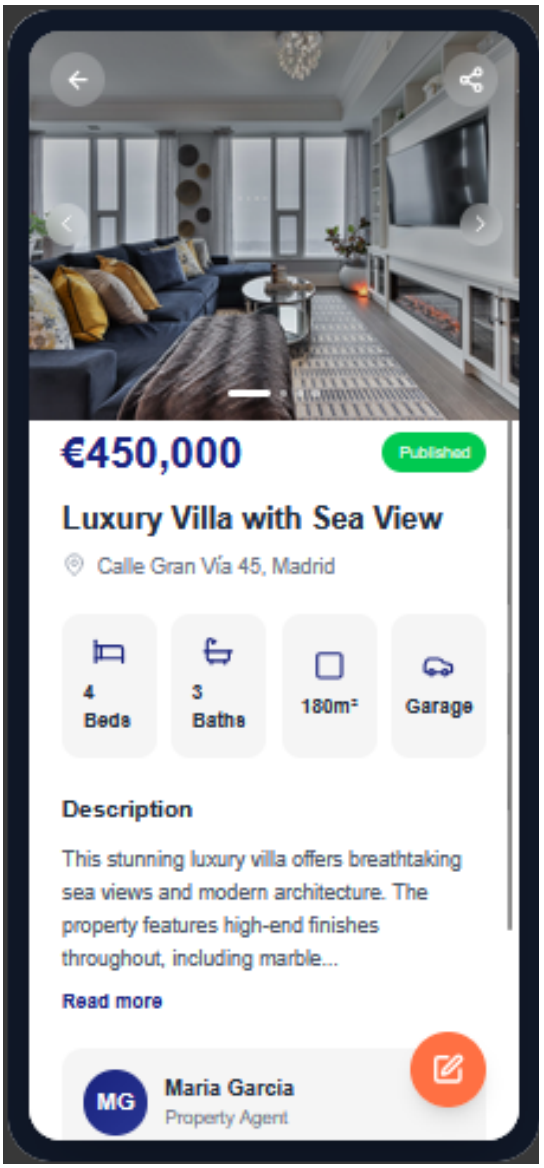
Pantalla de inicio de sesión que actúa como punto de entrada a la aplicación. La vista prioriza la identidad de marca en la parte superior y ofrece un formulario sencillo para introducir credenciales (email y contraseña). Incluye además enlaces de navegación secundaria para la recuperación de contraseñas y el registro de nuevos usuarios en la parte inferior de la pantalla.

10.2. Pantalla Principal de Inicio



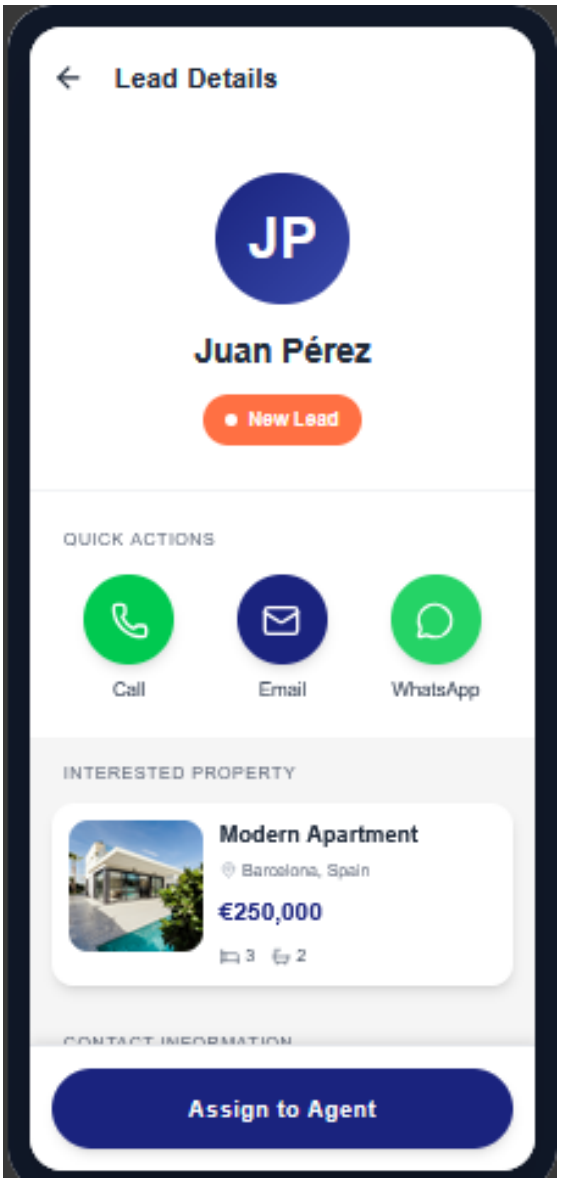
Pantalla de inicio que actúa como escaparate digital del catálogo inmobiliario. Presenta las propiedades disponibles mediante un listado vertical de tarjetas visuales, diseñadas para ofrecer una lectura rápida de datos esenciales como precio, ubicación y características principales. Incluye herramientas de búsqueda y filtrado en la cabecera para facilitar la localización de inmuebles específicos, garantizando una experiencia de navegación fluida e intuitiva para el usuario.

10.3. Detalles de la Propiedad



Vista de detalle diseñada para ofrecer un análisis exhaustivo del inmueble. La interfaz prioriza el contenido visual mediante un carrusel de imágenes inmersivo en la cabecera, seguido de una jerarquía clara de información que destaca el precio y el estado actual. Los atributos técnicos (habitaciones, superficie, extras) se organizan en una cuadrícula de iconos para facilitar la lectura rápida, mientras que el pie de página integra la información de contacto del agente responsable y un acceso directo para la edición de los datos.

10.4. Detalles de un Lead



Vista dedicada a la gestión integral del cliente potencial (Lead). La interfaz centraliza la información del contacto, destacando su estado actual y facilitando la interacción inmediata a través de botones de acción rápida para llamadas o mensajería. Incluye una tarjeta de contexto que vincula visualmente el lead con la propiedad de su interés y finaliza con una llamada a la acción persistente en la parte inferior para gestionar la asignación del agente, optimizando el flujo de venta.